INNOVACODE SOFTWARE SOLUTIONS

Guía Oficial de Replicación de Trabajo: Backlog, Tareas Técnicas y Flujo Git en Sprint 1

Manual operativo paso a paso para Diego Barrera y Sergio Rodríguez — Cómo interactuar con el tablero de GitHub Projects, ramas, tareas técnicas interactivas y Pull Requests hacia develop.

Propósito de este documento: Tras la integración exitosa del **Carril 1 (HU-01 y HU-02)** en la rama develop por parte de Jhoan Borja, esta guía detalla con exactitud cómo deben proceder **Sergio Rodríguez (Carril 2)** y **Diego Barrera (Carril 3)** para consultar sus Historias de Usuario, marcar sus tareas técnicas interactivas en el tablero de GitHub, desarrollar en sus ramas y entregar sus Pull Requests sin bloqueos ni conflictos de código.

1. Asignación de Carriles en Sprint 1 (22 Story Points)

El Sprint 1 está distribuido en 3 carriles paralelos e independientes para asegurar 0 colisiones en Git:

Carril / Responsable	Historias de Usuario	Pts	Rama Git Dedicada	Estado
Carril 1: Auth & Personal Jhoan Borja	• **HU-01:** Login Administrativo (3 pts) • **HU-02:** Gestión Personal (5 pts)	**8 pts**	feat/hu-01-02-auth-personal	**MERGED EN DEVELOP ✓**
Carril 2: Portal & Catálogo Sergio Rodríguez	• **HU-05:** Perfil y RETHUS (2 pts) • **HU-06:** Catálogo Servicios (3 pts) • **HU-07:** Catálogo Juguetes (3 pts)	**8 pts**	feat/hu-05-06-07-portal-catalogo	**EN PROGRESO**
Carril 3: WhatsApp & Cita Diego Barrera	• **HU-08:** Botón Flotante WhatsApp (1 pt) • **HU-09:** Formulario Cita Asistida (5 pts)	**6 pts**	feat/hu-08-09-whatsapp-cita	**EN PROGRESO**

2. Cómo Usar la Vista del Backlog en GitHub Projects

Todas las 22 Historias de Usuario ya cuentan con sus **Tareas Técnicas estructuradas con casillas interactivas**. Para visualizar y gestionar el avance diario:

Paso 1: Abrir el Tablero de Proyecto:
• Ingresa a GitHub en el repositorio: https://github.com/Borja-777/fonella-platfrom.
• Haz clic en la pestaña superior **Projects** y selecciona **Proyecto Fonella - Sprint 1**.

Paso 2: Abrir una Historia de Usuario (Panel Lateral):
• En la tabla o vista Kanban, haz un solo clic sobre el título de tu Historia asignada (ejemplo: *HU - Web Pública - 001*).
• Inmediatamente se desplegará el panel lateral derecho con la descripción completa, criterios Gherkin y observaciones.

Paso 3: Marcar Tareas Técnicas (Checkboxes Interactivos):
• Al final del panel verás la sección ### ■■ **Tareas Técnicas** con casillas interactivas ([ ]).
• A medida que completes cada archivo o endpoint en tu código local, **haz clic directo sobre la casilla**.
• ¡No necesitas editar el texto! GitHub la marcará automáticamente con un chulo (✓) y actualizará la barra de progreso general de la Historia.

3. Flujo Git Paso a Paso: Desde Develop hasta el Pull Request

Para garantizar que el código de Sergio y Diego se integre sin conflictos con la base ya completada por Jhoan:

Paso 3.1: Actualizar la rama local con develop

Antes de iniciar la jornada, descargar los últimos cambios de autenticación y endpoints subidos a develop:

```
cd fonella-platfrom
git checkout develop
git pull origin develop
```

Paso 3.2: Trabajar en la rama propia del Carril

Cambiar a la rama asignada e incorporar los cambios de develop mediante un rebase o merge limpio:

```
# Para Sergio (Carril 2):
git checkout feat/hu-05-06-07-portal-catalogo
git merge develop

# Para Diego (Carril 3):
git checkout feat/hu-08-09-whatsapp-cita
git merge develop
```

Paso 3.3: Ejecutar y Probar el Entorno Local

Levantar los contenedores de Docker y los servicios de backend y frontend:

```
# Terminal 1 - Base de Datos PostgreSQL:
docker compose up -d

# Terminal 2 - Backend NestJS (Puerto 4000):
cd fonella-backend
npm run start:dev

# Terminal 3 - Frontend Next.js (Puerto 3000):
cd fonella-frontend
npm run dev
```

Paso 3.4: Crear Commits Semánticos

Guardar los avances utilizando la convención estándar **Conventional Commits**:

```
git add .
git commit -m "feat(portal): implementar catalogo de servicios clinicos (HU-06)"
git push origin
```

Paso 3.5: Abrir el Pull Request hacia develop con Cierre Automático

1. Ve a GitHub: https://github.com/Borja-777/fonella-platfrom/pulls y haz clic en **New pull request**.
2. Configura las ramas exactamente así: **base: develop** ← **compare: tu-rama** (¡Nunca hacia main!).
3. En la descripción del PR, incluye la palabra clave Closes #X para cada historia terminada. Ejemplo:
   Carril 2: Closes #5 Closes #6 Closes #7
   Carril 3: Closes #8 Closes #9
4. **Asignar Revisor:** Asigna a **Jhoan Borja (Scrum Master / Software Architect)** como Reviewer para aprobación y merge.

4. Definition of Done (DoD) — Criterios de Aceptación Obligatorios

Una Historia de Usuario solo se considera terminada cuando cumple los siguientes 5 puntos:

1. **Criterios Gherkin Aprobados:** Se verificaron todos los escenarios descritos en la Historia de Usuario.
2. **Tareas Técnicas 100% Marcadas:** Todas las casillas [x] de la Issue están marcadas.
3. **TypeScript & Lint Limpios:** No existen errores de tipado (npx tsc --noEmit sale sin errores).
4. **Diseño UI Oficial:** Se utiliza estrictamente la paleta de colores de Fonella (Teal #136A7B, Turquesa #0AA896, Terracota #DF9B78) y tipografía nítida.
5. **PR Aprobado en Develop:** El código fue revisado por el Scrum Master y fusionado sin romper el build.